Compiled Memory

A Deterministic, Provenance-Traced Runtime for Long-Horizon AI Agent Context

Saber Maram

Statewave v1.0 · 2026

Contents

Abstract

Large language models are stateless functions of their context window. Every capability an agent appears to have "across time" — recognising a returning user, recalling a prior decision, not repeating a resolved mistake — is in fact a property of the *memory layer* that selects what enters that window, not of the model itself. The dominant memory architectures in 2026 treat this as an information-retrieval problem: embed the conversation, retrieve the top-*k* chunks by similarity, optionally extract facts with a second model, and hand the agent an opaque string. This paper argues that retrieval quality, while necessary, is the wrong primary objective for agent memory. Three properties that production and regulated deployments often require — **determinism** (the same query against the same snapshot returns the same bytes), **provenance** (every asserted fact traces to the immutable event that produced it), and **governability** (an operator can prove and constrain what state an agent saw at decision time) — are difficult to obtain from, and in some cases at odds with, the vector-first and graph-first designs that currently dominate.

We present **Statewave**, an open-source memory runtime organised around a four-stage lifecycle — RECORD → COMPILE → CONTEXT → GOVERN. Raw events are stored as immutable, append-only *episodes*; a pluggable compiler distils them into typed, confidence-scored *memories* with explicit validity windows and a citation chain back to their source episodes; and a deterministic assembler packs those memories into a token-bounded *context bundle* using a transparent, non-learned scoring function. We formalise the long-horizon context-assembly problem, give the exact scoring and packing model, and report a head-to-head, in-harness evaluation on **mem0's own benchmark suite** (a fork of `mem0ai/memory-benchmarks`, Apache 2.0; mem0's judge and scoring code untouched). On **LoCoMo** (Long Conversational Memory, n = 1,540) Statewave scores **0.905** vs mem0 cloud's **0.899** and mem0 OSS's **0.866**; on the smaller **LongMemEval** matched set (n = 30) the ordering holds at **0.967** / **0.933** / **0.833**. All three systems use the same `gpt-4o` answer and judge and a top-200 retrieval cap — only the memory backend differs across columns. We discuss what this implies for the design of memory systems, and we are explicit about where Statewave is still unproven (single run; LongMemEval per-category swings on n = 5 cells; BEAM pending).

---

1. Introduction

1.1 The memory problem is an infrastructure problem

An LLM agent is, at the level of its API, a pure function: `response = model(prompt)`. It has no hidden state that survives between calls. When such an agent appears to "remember" a customer across a three-week support thread, what actually happened is that some

surrounding system re-derived the relevant history and spliced it into the prompt. The intelligence of an agent over a long horizon is therefore bounded not by the model's parameters but by the quality of the process that decides *what the model gets to see*.

This is widely understood in the abstract and widely under-engineered in practice. The common failure mode is to treat memory as a thin convenience: dump the transcript into a vector store, retrieve a few similar chunks at query time, and hope the answer is in there. This works for demos but breaks down for the workloads where memory matters most — returning-customer support, multi-session coding agents, account-level sales copilots — precisely the workloads where the *history is long, the relevant facts are sparse, and being wrong is expensive*.

1.2 Retrieval quality is necessary but not sufficient

The research and product conversation around agent memory has converged on a retrieval framing: the goal is to surface the most relevant past content for the current turn. Benchmarks like LoCoMo measure exactly this, and improving it is genuine progress. But three requirements that are non-negotiable in serious deployments do not reduce to retrieval quality, and are in tension with the architectures that optimise for it:

1. **Determinism.** If the same agent, asked the same thing about the same subject at the same point in time, can receive different context on two different calls, then memory is not a *testable* surface. You cannot reliably write a regression test that says "after this code change, the billing agent still recalls the customer's plan." Vector-only retrieval does not generally provide such a guarantee: results can drift with index state, approximate-nearest-neighbour parameters, and reranker stochasticity.
2. **Provenance.** When an agent asserts "this customer is on the Enterprise plan and reported an SSO failure on the 12th," a security reviewer, a regulator, or an on-call engineer will ask *how do you know, and where did that come from?* If the fact is a substring inside an embedded chunk, the honest answer is "a similarity score put it near the query." That is not an audit trail.
3. **Governability.** An operator must be able to constrain and prove what an agent saw. *Can a marketing-facing tool read memories tagged PII? What exact state shaped the bundle the agent acted on? If a policy was wrong, what would the agent have seen under the corrected policy?* These are operator questions, not retrieval questions.

1.3 Thesis and contributions

This paper's thesis is that agent memory should be designed as a **compiled, governed runtime**, not as a retrieval index — and that doing so yields determinism, provenance, and governability as structural properties rather than bolt-ons, *without* sacrificing retrieval quality. Concretely we contribute:

- **A formalisation** of the long-horizon context-assembly problem and the objects it operates over (Section 3).
- **The Statewave model** — a four-stage RECORD → COMPILE → CONTEXT → GOVERN runtime — including the exact, non-learned, deterministic scoring and token-bounded packing function, the compilation and conflict-resolution pipeline, and the provenance/audit/policy layer (Section 4).

- **A structured comparison** of the prevailing memory architectures (stateless, prompt-stuffing, RAG-over-messages, vector-plus-extraction à la Mem0, non-embedding model-reads-files memory à la memU, temporal knowledge graphs à la Zep/Graphiti, and agent-OS designs à la Letta/MemGPT), identifying which of the properties above each exposes as a first-class contract (Section 5).
- **A head-to-head empirical evaluation** on a fork of mem0's own benchmark harness (`mem0ai/memory-benchmarks`, Apache 2.0; judge and scoring unchanged), in which Statewave matches the managed mem0 cloud and beats mem0 OSS on LoCoMo (n = 1,540) and on the smaller LongMemEval matched set (n = 30) (Section 6).
- **An honest accounting** of limitations and threats to validity (Section 7).

A note on maturity, stated once and meant: Statewave v1.0 is its **first stable public developer release**. That is a statement about API stability, not a claim of general availability, production-hardening, or battle-testing. The empirical results are single-pass and directional unless explicitly described otherwise. The argument is about *architecture*, and the core architectural properties are intended to hold by construction under the assumptions stated in Section 4.3 and are covered by regression tests today; the operational maturity is a separate axis on which the project is deliberately modest (Section 7).

---

2. Background: why stateless models need an external state machine

A useful way to see the design space is to ask what an agent *is allowed to condition on* at turn t. Let the full interaction history for an entity be a sequence of events $E = (e_1, e_2, \dots, e_n)$ accumulated over many sessions. The model can only condition on a context window C_t of bounded size B tokens. The memory system is the function

$$C_t = \mathcal{A}(E, q_t, B)$$

that maps the (possibly enormous) history E and the current query q_t into a bounded context C_t. Every memory architecture is, at bottom, a different choice of $\mathcal{A}$. The design question is not "should we have memory" but "what are the properties of $\mathcal{A}$" — is it deterministic, is its output explainable, can it be constrained, and how does its quality scale as $|E| \gg B$?

The hard regime is exactly $|E| \gg B$: a support customer with 10–50+ sessions, a coding agent with months of repo history. Here $\mathcal{A}$ must be lossy *by design*, and the interesting engineering is in *how* it loses information — arbitrarily (truncation), opaquely (a reranker), or in a principled, inspectable way.

---

3. Problem formulation

We make the objects explicit, because the differences between memory systems are differences in these objects.

Episodes. An episode $e = (\text{id}, s, \text{source}, \text{type}, \text{payload}, \tau, \dots)$ is an immutable, timestamped record of one raw interaction event for subject s: a conversation turn, a tool call, an

observation, a connector-ingested ticket. Episodes are append-only and never mutated. They are the *ground truth* of what happened.

Subjects. A subject s is the entity memory is *about* — a customer, an account, a workspace, a repository (`repo:<owner>/<repo>`), a user (`user-<id>`). All episodes and memories are scoped to a subject. Crucially, the subject is *not* the agent; multiple agents can read and write the same subject's memory.

Memories. A memory $m = (\text{id}, s, k, \text{content}, \text{conf}, v_{\text{from}}, v_{\text{to}}, \text{src}, \dots)$ is a *derived* unit produced by compiling one or more episodes. It carries a *kind* $k \in \{\texttt{profile\_fact}, \texttt{procedure}, \texttt{episode\_summary}, \texttt{artifact\_ref}\}$, a confidence score, a validity window $[v_{\text{from}}, v_{\text{to}})$ (where v_{to} may be null = "currently valid"), and — this is load-bearing — a set src $\subseteq \{e.\text{id}\}$ of the immutable source episode IDs it was derived from. Memories are derived data; episodes are truth.

Context bundle. The output C_t is a structured bundle, not a string: ranked sections (identity facts → procedures → history → recent episodes), a prompt-ready `assembled_context` serialisation, a `token_estimate`, and a `provenance` block listing exactly which fact IDs, summary IDs, and episode IDs were included.

The assembly objective. Given a subject s, a task/query q, a token budget B, and the compiled memory store M_s at time τ, the assembler selects an ordered subset $S \subseteq M_s$ that maximises a relevance objective subject to the budget:

$$\max_{S \subseteq M_s} \sum_{m \in S} \text{score}(m, q, \tau) \quad \text{s.t.} \quad \sum_{m \in S} \text{tokens}(m) \leq B.$$

This is a 0/1-knapsack-shaped problem. Statewave does not solve it optimally; it sorts by score and greedily packs section by section (Section 4.3). What matters for our argument is the *shape* of `score(·)` and the fact that it is a **fixed, transparent, non-learned function**, not a learned reranker. That single choice is what makes the whole system deterministic and testable.

Two properties we will hold the system to. Define $\mathcal{A}$ to be:

- **Deterministic** if and only if $\mathcal{A}(M_s, q, B)$ evaluated twice against the same store snapshot returns byte-identical output. Formally, $\mathcal{A}$ is a pure function of $(s, q, B, M_s^{(\tau)})$.
- **Provenance-complete** if and only if for every asserted unit $m \in S$ there is a finite, queryable chain $m \to \text{src}(m) \to \{e_i\}$ terminating in immutable episodes.

We show that Statewave satisfies both by construction under the assumptions stated in Section 4.3. Whether a given alternative satisfies them depends on its architecture and deployment; Section 5 discusses where each does and does not expose these as documented contracts.

---

4. The Statewave model

Statewave is a single self-hosted service (FastAPI) over PostgreSQL with the `pgvector` extension as its *only* storage dependency. Its lifecycle is four stages:

```
RECORD  →  COMPILE  →  CONTEXT  →  GOVERN
```

4.1 RECORD — episodes as immutable truth

Episodes arrive via the REST API (`POST /v1/episodes`, single or batched up to 100), the Python and TypeScript SDKs, or connectors (GitHub history, Markdown/ADRs, MCP agents, Slack, Zendesk, Intercom, etc.) that normalise foreign events into the native episode shape. Episodes are append-only and deduplicated by a stable `idempotency_key`. There is no update or in-place edit path. This is the foundation of provenance: because the raw record is immutable, a citation to it cannot rot.

4.2 COMPILE — distilling typed, scored, provenance-linked memory

Compilation is an explicit, idempotent step (`POST /v1/memories/compile`) that reads *uncompiled* episodes for a subject and emits typed memories. Two compilers ship:

- **Heuristic compiler** (default): regex/pattern extraction of profile facts, preferences, procedures, and per-episode summaries. Zero external calls, fully local, deterministic output, confidence in the 0.6–0.8 band. This is the default for a reason: zero data egress, zero API cost, stable behaviour.
- **LLM compiler**: routes episode batches (≤ ~6000 chars/batch, ≤ 4 in-flight requests, run in a thread pool so as not to block the event loop) through LiteLLM to any provider — OpenAI, Azure, Anthropic, Bedrock, Cohere, or a self-hosted Ollama/vLLM model for fully local operation. It extracts richer, implicit structure. On any provider or parse error it **falls back to the heuristic compiler** — compilation never silently drops episodes.

Both write to the same `memories` table with identical schema and provenance. After raw extraction, each candidate memory is embedded (a deterministic stub provider for dev/test, or a real LiteLLM embedding model) and passed through **conflict resolution**: within a `(subject, kind)` group, Jaccard word-overlap above a threshold marks the older memory as superseded by setting its v_{to}, rather than deleting it. The superseded memory remains in the store, queryable, with its provenance intact — temporal lineage is preserved, not overwritten. Extraction, embedding, and conflict resolution commit in a single database transaction.

Idempotency is a property, not a hope: recompiling a subject processes only episodes not yet compiled and inserts nothing already present. A regression test asserts that a recompile produces zero new memories.

4.3 CONTEXT — deterministic, token-bounded assembly

This is the core of the determinism claim, which holds under the explicit assumptions stated at the end of this section. Assembly proceeds in two phases.

Phase 1 — policy filter (the first gate). Before any scoring, each candidate memory is evaluated against the active sensitivity-label policy bundle using the call's `caller_id`/`caller_type`. Memories matching a `deny` rule under `policy_mode: enforce` are *removed from the candidate pool entirely* — they cannot leak through via a high relevance score. Memories matching a `redact` rule are retained but their content is replaced with `[REDACTED by policy]`. Either way the decision is recorded (Section 4.4). Filtering before scoring is deliberate: governance is not something a good relevance score can override.

Phase 2 — deterministic composite scoring. Each surviving candidate is scored by *summing* a small set of bounded signals. The core signals, always applied:

Signal	Range	Definition
Kind priority	3–10	`profile_fact`=10, `procedure`=8, `episode_summary`=5, `raw_episode`=3
Recency	0–5	Linear across the candidate set; most recent gets the max
Task relevance	0–5 (lexical) or 0–8 (semantic)	Word-overlap fraction with the task, or cosine similarity if embeddings exist
Temporal validity	−4 to +3	Currently valid (`valid_to` null or future) = +3; expired = −4

For support-agent workloads, additive adjustments nudge (not replace) the core score: an active-session boost (+6), a resolved-session penalty (−5), open-issue (+4), action-step (+2), urgency (+2), idle-chatter (−2), and a repeat-issue boost (+4, or +6 if the prior issue was resolved). The numeric scales are chosen so that no single signal can dominate the kind-priority ceiling — one strong match cannot single-handedly evict a high-priority identity fact.

So the score is, schematically:

```
score(m, q) =  priority(kind(m))
             + recency(m)                       # 0..5, linear over candidate set
             + relevance(m, q)                  # 0..5 lexical  | 0..8 semantic
             + validity(m)                      # +3 valid | -4 expired
             + Σ support_adjustments(m, q)      # additive, bounded
```

Phase 3 — ranked greedy packing. Candidates are sorted by descending score and packed section by section (facts → procedures → history → episodes), reserving 10 tokens of header per section, until the budget B is exhausted. Items are not truncated mid-content; the assembler stops including whole items once the budget would be exceeded. If embeddings are unavailable, the semantic term gracefully drops out and lexical overlap is used — the system degrades, it does not fail.

The scoring weights are constants in the service (`server/services/context.py`); they are *not* per-tenant or per-call configurable today. This is a deliberate, defended choice: configurability is not shipped speculatively, because a tuning surface that nobody has shown a real misranking for is just a way to break determinism guarantees. The escape hatches are honest ones — pre-filter the candidate set by `kind`/`subject`, or subclass the assembler in your own deployment.

Determinism property under stable inputs. Under a fixed set of inputs — a fixed memory-store snapshot $M_s^{(\tau)}$, a stable policy bundle, a stable caller identity, a stable serialisation format, and deterministic tie-breaking for equal scores (an explicit, stable sort key rather than reliance on database row order) — assembly is a pure function of $(s, q, B, M_s^{(\tau)})$ and returns byte-identical output on repeated calls. This holds because (i) the policy filter is a pure function of the candidate, the policy bundle, and the caller identity; (ii) the score is a fixed arithmetic function of stored signals; and (iii) packing is a deterministic sort-then-greedy over that score with ties broken by the stable key.

Two assumptions are worth making explicit. First, when semantic scoring is enabled the relevance term depends on stored embedding vectors; determinism then requires those vectors to be fixed in the snapshot — which they are, because embeddings are computed once at COMPILE time, not per retrieval. Second, the guarantee is about *retrieval*: the COMPILE step may call an external model and is not itself deterministic across model or provider versions, which is one reason episodes — not compiled memories — are treated as ground truth. Within these assumptions, retrieval introduces no approximate-nearest-neighbour or learned-reranker variance, which is what makes the assembled context regression-testable — a meaningful difference from retrieval paths built on learned rerankers or approximate-nearest-neighbour search.

4.4 GOVERN — provenance, receipts, and policy

The fourth stage — governance — is comparatively uncommon among the memory systems we reviewed.

Provenance as a finite query. "Why did the agent say X?" resolves to a finite chain: `bundle.provenance.fact_ids` → `memories` → `source_episode_ids` → `episodes`. Every link is stored; none requires forensic reconstruction. Because episodes are immutable, the terminus is stable.

State-assembly receipts. Any `/v1/context` or `/v1/handoff` call can emit an immutable, ULID-addressable *receipt* recording exactly which memories and episodes shaped the bundle, plus a SHA-256 hash of the bytes delivered to the agent. Receipts answer the compliance question *"what state did the agent actually see at decision time?"* without trusting application logs. Emission is gated by a kill-switch → per-tenant config (`always | on_request | never`) → per-request flag. Receipts can be HMAC-signed (`hmac-sha256-canonical-v1`, operator-held keys never persisted to the DB) and verified in constant time via `GET /v1/receipts/{id}/verify`. And they are *replayable*: each v0.9+ receipt embeds the active policy YAML, so `POST /v1/receipts/{id}/replay` re-runs against current memories under the original policy and returns a structural diff — *current code, original policy* — for "what changed since the agent acted?" analysis.

Declarative policy. Per-memory `sensitivity_labels` feed a content-hashed, immutable YAML/JSON policy bundle with predicates and `deny`/`redact` actions, first-match-wins, default-allow. A `log_only` mode records decisions into receipts *without* filtering — so operators can roll out a policy and observe its effect before switching to `enforce`. An opt-in heuristic auto-labeller stamps advisory `suggested_labels` (`pii.email`, `pii.phone`, `financial.card` via Luhn, `secret.token`), kept strictly separate from the authoritative labels until an operator promotes them through a review endpoint.

Lifecycle controls. Delete-by-subject (`DELETE /v1/subjects/{id}`) gives GDPR-style erasure; a scheduled retention-purge worker tombstones receipts past their configured retention; per-tenant data residency can hard-pin a tenant to a region with 403 on mismatch.

4.5 Portability and access

Episodes come in via SDKs or connectors; memory comes out via `/v1/context`, the typed SDKs, or an **MCP server** that exposes Statewave memory to any MCP-compatible client (Claude, Copilot, Cursor). A `.swmem` archive format moves a subject's memory between instances and

environments. Statewave does *not* run the agent loop, call your LLM, or route tool calls — it does one job: *give me the right, governed, explainable context for this subject and this task*.

---

5. A structured comparison of memory architectures

This section positions Statewave relative to the prevailing memory architectures. The aim is not to rank products but to identify, for each design, which of the properties from Section 1.2 it exposes as a documented, first-class contract — and which it leaves to the application layer or a custom deployment.

Scope of comparison. This comparison is based on public documentation, public APIs, and observable product surfaces reviewed for the 2026 publication draft. It does not claim that competing systems could not implement similar controls internally, through custom deployments, or in future releases; several are open-source and actively evolving. The comparison concerns the *documented first-class contracts* exposed to operators and developers, not the limits of what each system could be made to do. Note also that the benchmark in Section 6 measures Statewave directly against mem0 cloud and mem0 OSS on a fork of mem0's own harness; statements here about Zep, memU, Letta, and other systems are qualitative, based on their documented design, and are not backed by a head-to-head run in this draft.

5.1 Stateless prompting

The agent sees only the current message — no identity, no history. By construction it cannot serve multi-session workloads, since there is no state to carry across turns. A useful baseline and nothing more.

5.2 Prompt stuffing (full-history replay)

Concatenate the whole transcript. This struggles in the $|E| \gg B$ regime: token cost scales linearly with subject lifetime, there is no ranking (critical facts compete with chatter), no provenance, and context-window limits eventually force truncation. It works until the history outgrows the window, after which content is dropped without ranking.

5.3 RAG over raw messages

Embed every message; retrieve top-*k* by similarity. This is the field's default, and on its own it does not provide the properties from Section 1.2: retrieval is typically non-deterministic (results can drift with index state and ANN parameters); there is no structured extraction by default ("Alice is on Enterprise" remains a substring inside a chunk); no confidence scoring or temporal reasoning (a superseded email can rank equal to the current one); and no built-in provenance (a chunk is not linked to the interaction that produced it). Token budget is managed by truncation rather than ranked priority. It is very effective at "find similar text"; it is simply not designed, by itself, to be an auditable, testable memory of record.

5.4 Vector + LLM extraction (Mem0 and the structured-memory wave)

A large and fast-moving class — Mem0, and a 2026 wave of open-source frameworks (e.g. Memori, LangMem) — improves on raw RAG by adding an LLM extraction pass: facts are pulled out of messages and stored, often over a vector store with a graph layer and a reranker.

This genuinely helps, and on recall it is the strongest of the mainstream approaches (Section 6 reports the managed mem0 cloud at near-parity with Statewave overall on both LoCoMo and LongMemEval). What its public interface does not appear to expose as a first-class contract:

- **A documented determinism contract.** The reranker and managed retrieval stack are not described as reproducible in the public documentation reviewed, and the composition is not exposed, so the exact assembled context is not straightforward to regression-test.
- **A surfaced provenance chain.** The public surface does not appear to expose a queryable citation from each extracted fact back to the specific source events.
- **A token-bounded contract.** A first-class `max_tokens` request parameter with a reported estimate against budget does not appear to be part of the model; budget is approximated through a `top_k`-style proxy.
- **Self-hosted-only data control.** A managed plane is the headline product, with hosted reranker/graph services performing retrieval (Mem0 also offers an open-source self-hosted option).

Many of these frameworks are strong at what they optimise for — fast personalization with minimal ops. The narrower point is that they optimise primarily for recall, and that auditability and reproducibility are different objectives that they do not appear to target as first-class contracts.

5.5 Non-embedding, model-reads-memory (memU)

A distinct and instructive design, announced on Reddit's r/LocalLLaMA subreddit in late 2025, is **memU** (NevaMind-AI) — an agentic memory framework that deliberately rejects embedding similarity as the *primary* retrieval mechanism. Its thesis ("non-embedding search") is that LLMs are already good at reading structured text, so rather than force every recall through vector similarity, the model should read **structured memory files directly**. memU organises memory in three layers — a *resource layer* (raw text/image/audio/video), a *memory item layer* (extracted fine-grained facts/events), and a *memory category layer* (themed files the model reads) — and the store **self-evolves**: frequently-accessed memory is promoted, rarely-accessed memory fades, with no manual pruning.

memU and Statewave **agree on the most important negative claim in this paper**: embedding top-*k* should not be the primary retrieval mechanism for agent memory, and raw events should be compiled into structured, typed/categorised memory. memU's three layers map almost directly onto Statewave's episodes → memories → ranked sections, and the convergence is itself evidence that the field is moving past naive RAG. The divergence is precisely the three properties this paper is about:

- **Determinism.** memU's self-evolution makes retrieval *path-dependent*: what the model sees depends on the access history that promoted or faded each file. Two subjects with identical content but different usage histories may retrieve differently, and the same subject may retrieve differently over time as the store reorganises itself. This makes it difficult to offer the fixed-snapshot determinism contract of Section 3. Statewave's store also changes over time, but only by *append and explicit supersede-with-`valid_to`* — not by usage-driven reorganisation — so ranking at a fixed snapshot is reproducible.
- **Retention vs. fading.** "Rarely-accessed memory fades out" is ergonomic for a personal assistant, but a different trade-off for a memory intended as a system of record: a fact

that was never asked about is not a fact that is false, and for compliance and support the rarely-touched detail (the single SSO failure six months ago) is often the one that matters. Statewave supersedes but does not delete — the superseded row stays queryable with provenance — so retention does not depend on access frequency.

- **Provenance, audit, token contract.** With the model reading files directly, the public surface reviewed does not define a token-bounded packing contract (the model reads whatever files it elects to), a surfaced per-fact citation chain to immutable source events, or a receipts/policy gate. memU targets lightweight, self-organising, multimodal personal memory rather than deterministic, auditable, governed retrieval.

In short, memU is close to Statewave *in spirit* — both decline embedding-only retrieval and compile raw events into structured memory — and differs mainly in its embrace of self-mutating, model-curated memory. The two optimise for different goals: memU for an ergonomic, self-organising store; Statewave for a store that stays fixed under a snapshot so it can be tested, cited, and governed.

5.6 Temporal knowledge graph (Zep / Graphiti)

Zep models memory as a dynamically-updated graph of entity nodes and fact edges, with valid/invalid timestamps on relationships, and returns an opaque "Context Block" string. This is the right shape for genuinely relational questions ("trace the path from this user through three accounts to that incident") and gives temporal validity on relationships for free. Its costs relative to our three properties: retrieval is graph-traversal plus reranking (not described as deterministic in the materials reviewed), the high-level return is an opaque "Context Block" string rather than per-fact inspectable metadata, and it adds a graph store as operational surface. For the *typed-fact* questions that dominate support / coding / sales agents ("what's their plan, what's open, what did they say last week?"), a graph adds operational surface those workloads may not need, and deterministic, per-fact-inspectable retrieval is not the design centre of the public interface reviewed.

5.7 Agent-OS / self-managed memory (Letta / MemGPT)

Letta (continuing the MemGPT line) is an *agent runtime*: the model manages its own core/archival/recall memory inside an OS-like loop. This is a different layer of the stack, not a competitor at the memory-service layer — and a powerful one if you want a complete runtime. But its memory is LLM-managed (the model decides what to keep), which contrasts with the "compiled memories are records, not model decisions" stance here. It does not appear to offer a deterministic, externally-testable retrieval contract at the memory-service layer, because the loop's behaviour is governed by the model and its tool calls. If you already have an agent loop, you may not want to adopt a new runtime simply to gain memory; a memory layer your existing loop can call may fit better. The two are not directly comparable at the memory-service layer.

5.8 Summary of the gap

The matrix is split into two tables for readability. Both share the legend that follows.

Table A — Architecture / runtime properties

Property	Stateless	Prompt-stuff	RAG	Vec+extract	memU	Zep	Letta	**Statewave**
Scales to long-horizon history	✗	✗	~	✓	✓	✓	✓	✓
Deterministic contract	n/a	✗	✗	✗	✗	✗	✗	✓
Token-bounded contract	✗	✗	✗	~	✗	~	~	✓
Runs the agent loop	✗	✗	✗	✗	✗	✗	✓	✗

Table B — Governance / audit / deployment properties

Property	Stateless	Prompt-stuff	RAG	Vec+extract	memU	Zep	Letta	**Statewave**
Surfaced provenance chain	✗	✗	✗	✗	~	~	✗	✓
Auditable receipts	✗	✗	✗	✗	✗	✗	✗	✓
Policy-gated retrieval	✗	✗	✗	~	✗	~	✗	✓
Retains rarely-used facts	n/a	✓	✓	✓	✗	✓	~	✓
Self-hosted, single dependency	n/a	n/a	~	~	~	✗	~	✓

Column key: *Vec+extract* = Mem0 and similar vector-plus-extraction systems; *Zep* = Zep/Graphiti; *Letta* = Letta/MemGPT.

Legend: ✓ = documented as a first-class contract in the public interface reviewed; ~ = partially supported, possible with additional engineering, or not directly comparable; ✗ = not documented as a first-class contract in the public interface reviewed (which is not the same as impossible — see *Scope of comparison*). The matrix summarises *documented contracts*, not measured performance, and reflects the authors' reading of public materials; it is not a benchmark. The "Runs the agent loop" row is a feature of Letta, not a deficiency of Statewave — they sit at different layers; "retains rarely-used facts" is marked ✗ for memU because of its usage-based fade.

In summary, the mainstream systems are strong on *recall under* $|E| \gg B$ and generally do not expose *determinism, surfaced provenance, and governance* as first-class contracts in the interfaces reviewed. Statewave aims to provide the latter without conceding the former — a

claim Section 6 tests empirically against the two systems benchmarked here (mem0 cloud and mem0 OSS).

---

6. Empirical evaluation

6.1 Setup

We evaluate on **mem0's own benchmark suite** — a fork of `mem0ai/memory-benchmarks` (Apache 2.0). The fork keeps mem0's judge, scoring, and benchmark code unchanged; the additions are a Statewave retrieval adapter and a `--backend statewave` dispatch across the runners, plus a small set of harness fixes that **help the mem0 backends** (the cloud `add` URL, mem0 OSS message-content date grounding, BEAM ingest concurrency — all listed in the repo's `NOTICE`; see §6.3). Running on a competitor's own harness, on their own benchmarks, is a deliberate choice: it removes any "we wrote the test" objection.

Two long-term-memory benchmarks are reported here, with **BEAM** (long-context) pending:

- **LoCoMo** (Long Conversational Memory; Maharana et al.) — a public benchmark of long, multi-session conversational dialogue. **n = 1,540 questions** across four categories: single-hop, multi-hop, temporal, and open-domain.
- **LongMemEval** (Wu et al.) — a long-term-memory evaluation across six question types (knowledge-update, multi-session, single-session-assistant, single-session-preference, single-session-user, temporal-reasoning). The matched set used here is **n = 30 questions** (five per type), which has wide per-category error bars; LoCoMo is the more robust signal.

Each (system, question) is run through **ingest → search → answer → judge**. The judge scores the answer against the dataset's ground truth as CORRECT (1.0) or INCORRECT (0.0); the per-system overall score is plain accuracy across all questions. Both answerer and judge are LLMs; the model identity is recorded in every result file so a reproduction can confirm what was used.

The configuration of the run reported in this section is summarised below.

Item	Value
Benchmarks	LoCoMo (n = 1,540) · LongMemEval (matched set, n = 30) · BEAM pending
Answerer model	`gpt-4o`
Judge model	`gpt-4o`
Extractor / embedder (Statewave, mem0 OSS)	`gpt-4.1` extraction · `text-embedding-3-small`
Extractor / embedder (mem0 cloud)	mem0's managed extractor/embedder (not user-configurable)
Retrieval cutoff	**top-200** for all three systems (mem0 OSS's library default of 20 is documented in §6.3)
Run count	1 (single run per system per benchmark; multi-run aggregates are the next step)
Source results	`statewave-memory-benchmarks/results/statewave_comparison` (per-question JSONL per system × benchmark)

This is **not** a reproduction of mem0's own published numbers, which use `gpt-5` and Qwen-3 embeddings. Holding the answerer, judge, embedder, and retrieval cutoff constant across the three columns below is the point: the only thing that varies is the memory backend.

6.2 Results

LoCoMo (n = 1,540) — overall accuracy at the top-200 cutoff

System	Overall	Multi-hop	Open-domain	Single-hop	Temporal
Statewave	**0.905**	**0.943**	**0.688**	0.913	0.913
mem0 cloud (managed)	0.899	0.929	0.667	0.904	**0.928**
mem0 OSS	0.866	0.855	0.646	**0.916**	0.813

Statewave leads overall and on multi-hop and open-domain. mem0 cloud edges Statewave on temporal by 1.5 pts; mem0 OSS edges Statewave on single-hop by 0.3 pts. The overall ordering and the multi-hop margin are the load-bearing results; the per-category cells are reported in full for honesty rather than for headline emphasis. **Single run** — for an n = 1,540 benchmark this is informative, but is not a substitute for a multi-run aggregate with confidence intervals.

LongMemEval (matched set, n = 30) — overall accuracy at the top-200 cutoff

System	Overall	knowledge-update	multi-session	sng-sess-asst	sng-sess-pref	sng-sess-user	temporal
Statewave	**0.967**	**1.000**	**1.000**	**1.000**	**1.000**	0.800	**1.000**
mem0 cloud (managed)	0.933	1.000	0.800	0.800	1.000	**1.000**	1.000
mem0 OSS	0.833	0.800	1.000	0.600	0.800	1.000	0.800

(*sng-sess-asst* = single-session-assistant, *sng-sess-pref* = single-session-preference, *sng-sess-user* = single-session-user.) The matched set is small — **five questions per category** — so per-category swings are sensitive to a single answer; treat the per-category cells as *directional*. The overall ordering matches LoCoMo's: Statewave > mem0 cloud > mem0 OSS.

6.3 Methodological honesty

The fork keeps mem0's judge and scoring code untouched, but we did fix things in the surrounding harness that **helped the mem0 backends**. The material changes (per the repo's `NOTICE`):

- **mem0 cloud `add` URL.** Upstream pointed at a v1 endpoint that ingested nothing in the cloud account we tested against; the fork uses the documented v3 `add` path. Without this fix the cloud column would have been a degenerate baseline.
- **mem0 OSS v2 search filter semantics.** Upstream used the v1 filter shape; v2 changed how user-scoped filters are passed. The fix lets OSS see only the matching user's memories — the filter behaviour their docs describe.

- **Session-date grounding for mem0 OSS.** The mem0 OSS SDK strips the per-session date from the message metadata; we restore it so OSS receives the same temporal grounding the other systems do.
- **BEAM ingest concurrency.** Upstream was effectively single-threaded; the fix parallelises the ingest loop. (BEAM results are pending.)

We list these because if the reaction to "Statewave matches the paid mem0 cloud" is "compared with which version of mem0 cloud?" the answer is in the diff. Apart from these harness fixes and the Statewave adapter itself, mem0's judge and scoring code are unmodified.

A second product-inherent asymmetry the fork **does not** equalise: **mem0 OSS's SDK default returns ≤ 20 memories per query**, whereas Statewave and mem0 cloud naturally return ~200 in this setup. All three columns in §6.2 are reported at top-200 for consistency; the cleanest side-by-side is Statewave vs mem0 cloud, both at their natural top-200 ceilings.

Benchmark scores are reproducible evaluation signals, not universal claims. They move with the answerer, the judge, the extractor, the embedder, the retrieval cutoff, the seed, and the runtime environment, and should only be compared across systems within the same configuration.

6.4 Workload-level evidence

Beyond LoCoMo, the support-agent workflow is checked by a runnable, CI-friendly eval suite — three suites totalling 56 binary assertions (context-quality 7 tests/15 assertions, handoff 6/15, advanced 10/26) — covering identity persistence across sessions, task-relevant ranking, session-awareness, repeat-issue surfacing, explainable health scoring, deterministic handoff packs, and provenance. A workflow benchmark scores Statewave 8/8 against a naive stateless baseline's 2/8 on active-issue handling, repeat detection, health, provenance, and resolution-ranking. These are property tests, not accuracy points — they assert the *contract* (e.g. "recompile produces zero new memories," "identical requests produce identical handoff notes and score") holds.

---

7. Limitations and threats to validity

We list these explicitly, because they bound the claims made above.

Empirical. - **Both reported benchmarks are single runs.** The answerer and judge are stochastic, so per-category swings — especially on LongMemEval's n = 30 matched set, where each category is five questions — should be read as directional. The right next step is a multi-run aggregate with confidence intervals on both LoCoMo and LongMemEval; we have not yet produced one in this harness. - **BEAM (long-context) is in the harness but not reported here.** That benchmark is what would most directly stress the *long-history* regime against which Statewave's compile-then-retrieve design is meant to pay off; running it on the same fork is straightforward and will follow. - **The LLM-extractor configuration was used** (Section 6.1: `gpt-4.1` extraction + `text-embedding-3-small` embeddings). We have **not** published a comparative quality eval of this configuration against Statewave's heuristic compiler (the local-only default for deployments without an LLM key), nor a 50-session production-scale benchmark.

Architectural / operational. - **Single-Postgres only.** Multi-replica API is supported and verified since v0.8 (Fly multi-machine, Helm HPA), but cross-region / multi-Postgres clustering is roadmap, not shipped. - **Multi-tenant isolation is app-layer** query scoping, not Postgres row-level security; not battle-tested at scale. - **Not load-tested** beyond modest scale (>10k subjects, high throughput is unproven). - **No graph reasoning.** Genuinely relational, multi-hop-across-entities questions are not Statewave's shape — that is closer to Zep's domain. - **No managed plane.** Self-hosted only; SOC 2/HIPAA-style compliance programmes are the operator's responsibility (Statewave supplies the technical controls — local heuristic path with zero egress, delete-by-subject, receipts — not the audited service). - **Scoring is not configurable** today; if your workload misranks under the fixed weights, that is a real limitation (with the subclass/pre-filter escape hatches noted in 4.3).

If any of these are blockers, Statewave may not be the right fit for that use case yet.

---

8. Discussion: design principles

Three principles fall out of the architecture and the evaluation.

1. **Raw truth first, derived memory second.** Episodes are immutable and primary; memories are derived and disposable (recompilable). This inverts the RAG default (store text, derive nothing) and the agent-OS default (let the model curate its own memory). It is what makes provenance a query and idempotency a property.
2. **Determinism is a design choice with real trade-offs.** Adding a learned reranker to the retrieval path can improve recall in exchange for testability. For memory — a surface that both a regulator and a regression suite may interrogate — a fixed, transparent scoring function is, we argue, often the better trade. The escape hatches remain (pre-filter or subclass), whereas nondeterminism introduced by a reranker is harder to remove after the fact.
3. **The compiler is the recall lever; the budget is the cost lever.** In a compile-then-retrieve architecture the retrieval budget governs *how much* of the answerable set is shown, while the compiler — its extraction coverage, conflict resolution, and typing — governs *what the answerable set is*. The head-to-head numbers in Section 6 hold the retrieval cutoff fixed and move the backend; the orthogonal experiment — fixing the backend and sweeping the cutoff — is the future-work direction that would let this trade-off be measured end-to-end against the stochastic noise of a single answerer/judge pair. We name the principle here so future evaluations can target it explicitly rather than incidentally.

---

9. Conclusion

Agent memory is the function that decides what a stateless model gets to condition on over a long horizon. The field has, reasonably, optimised that function for retrieval quality. This paper argued that three further properties — determinism, provenance, and governability — are requirements rather than niceties for serious deployments, and that they are not generally exposed as first-class contracts by vector-first and graph-first designs. Statewave reframes

memory as a **compiled, governed runtime**: immutable episodes, idempotently compiled into typed, scored, provenance-linked memories, assembled by a transparent deterministic function into token-bounded, auditable context bundles, behind a policy gate. The head-to-head evaluation in Section 6 — run on a fork of mem0's own benchmark harness with the judge, scoring, and answer/judge models held constant — indicates that this architecture need not trade away recall to obtain those properties: Statewave matches the managed mem0 cloud and beats mem0 OSS on LoCoMo (0.905 / 0.899 / 0.866) and holds the same ordering on LongMemEval (0.967 / 0.933 / 0.833). The architectural thesis is what carries beyond any single benchmark.

Statewave v1.0 is a first stable public developer release, and this paper is as explicit about what is unproven as about what is shown. But the architectural thesis stands independently of the project's maturity: if your agents must remember reliably, explainably, and under control — not merely plausibly — then memory should be compiled and governed, not merely retrieved.

References

1. Maharana, A. et al. *LoCoMo: Evaluating Very Long-Term Conversational Memory of LLM Agents.* Snap Research. https://github.com/snap-research/LoCoMo
2. Packer, C. et al. *MemGPT: Towards LLMs as Operating Systems.* 2023. https://arxiv.org/abs/2310.08560 (the research lineage behind Letta).
3. Mem0. *The memory engine for AI.* https://docs.mem0.ai/overview · https://github.com/mem0ai/mem0
4. Zep / Graphiti. *Graph-RAG temporal-knowledge-graph memory.* https://help.getzep.com/concepts · https://github.com/getzep/zep
5. Letta. *Stateful agents that remember, learn, and improve.* https://docs.letta.com · https://github.com/letta-ai/letta
6. memU (NevaMind-AI). *Non-embedding agentic memory framework; the model reads structured memory files directly across resource / item / category layers, with usage-based self-evolution.* https://github.com/NevaMind-AI/memU · https://app.memu.so
7. LiteLLM — unified LLM/embedding provider gateway. https://github.com/BerriAI/litellm
8. Statewave architecture overview, ranking model, compiler modes, sensitivity labels, and receipts (in-repo): `statewave-docs/architecture/{overview,ranking,compiler-modes}.md`, `statewave-docs/{why-statewave,receipts,sensitivity-labels}.md`.
9. Statewave Memory Benchmarks — head-to-head LoCoMo / LongMemEval evaluation against mem0 cloud and mem0 OSS, with per-question result JSONLs: https://github.com/smaramwbc/statewave-memory-benchmarks (`results/statewave_comparison/`). Forks `mem0ai/memory-benchmarks` (Apache 2.0): https://github.com/mem0ai/memory-benchmarks · the `NOTICE` file documents every harness change against the upstream commit.
10. Comparative analyses (in-repo): `statewave-docs/comparisons/{mem0,zep,letta,langchain-memory,`

Prepared against Statewave v1.0 and competitor documentation as of 2026. Benchmark numbers come from a fork of `mem0ai/memory-benchmarks` (Apache 2.0), with mem0's judge and scoring code unchanged and the harness changes documented in the repo's `NOTICE`. LoCoMo (n = 1,540)

and LongMemEval (matched set, n = 30) each ran once per system at the configuration described in §6.1; multi-run aggregates are the right next step and supersede these when complete. Competitor feature sets, pricing, and licensing move — verify against live documentation before relying on the comparison.